



Overview

Scoring looks like the easiest part of a game, and it is where students first meet a real design question: *who owns the score, and how does everyone else find out about it?* The chapter answers it in three steps. `ch06_hud_events` shows three Phaser emitters (a scene's, the game's, the registry's) feeding a HUD scene, and what each does when a scene restarts. `ch06_coin_collector` puts the rules and numbers in a plain `ScoreManager` class that **announces** changes as events, with a HUD scene that only listens. `ch06_high_score_table` saves a top-ten table in `localStorage`, validates what it reads back, and takes initials from the keyboard.

The Phaser ideas are **parallel scenes** (`launch` / `stop`, draw order), **event emitters** (`emit` / `on` / `once` / `off`, context, `changedata-`), **counter tweens** and generic `keydown` events. The TypeScript and JavaScript ideas are `extends Phaser.Events.EventEmitter`, `unknown` and **type guards**, `try / catch`, `JSON`, optional properties and optional chaining, `padStart`, and regular expressions. The OO idea - tell an object what happened, let it announce the results - is the one to leave them with.

Prerequisites

- Chapter 2: scenes, `init(data)` with an interface, the registry, the scene object being reused
- Chapter 4: `launch`, parallel scenes, `SHUTDOWN` and cleaning up listeners (the chapter recaps what it needs in a few lines, so it can be taught before Chapter 4 at a push)
- Java: interfaces, inheritance, the observer pattern / Swing listeners (for the analogy)

Learning outcomes

Students can:

1. run a HUD as a separate scene on top of the game, and explain why it is better than text in the game scene
2. send and listen for events on a scene, on the game and on the registry, and choose between them
3. explain why a late listener misses events, and read the current state as well as listening
4. remove listeners from longer-lived emitters on `SHUTDOWN` , passing the same function and context
5. design a plain class that owns game state and emits events, and keep scenes to *telling* and *listening*
6. save and load structured data with `localStorage` and JSON, and validate it with `unknown` , `try / catch` and a type guard
7. read typed characters from `keydown` events with `event.key`

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Play <code>ch06_coin_collector</code> . Ask: "what is on screen that is not the game?"
0:05 - 0:15	Slides 3-4: why a HUD scene; <code>launch</code> , draw order, each scene's own camera (point out the shake does not move the HUD)
0:15 - 0:30	Slides 5-9 with <code>ch06_hud_events</code> running: the three routes. Live demo (below) of the missing clean-up
0:30 - 0:45	Slides 10-12: the <code>ScoreManager</code> , tell vs announce. Draw the "who talks to whom" diagram on the board before showing it
0:45 - 0:55	Slides 13-17: floating text and count-up; <code>localStorage</code> , JSON and validation
0:55 - 1:00	Slide 18: initials; the challenge list

Lab (2 hours)

Time	Activity
0:00 - 0:15	Build and play all three projects. In <code>ch06_high_score_table</code> , get onto the table, then find the key in the browser's developer tools (Application > Local Storage in Chrome)
0:15 - 0:30	Break it on purpose: edit the stored value to <code>this is not json</code> , reload - the game still starts. Then comment out the <code>try / catch</code> round <code>JSON.parse</code> and reload
0:30 - 1:15	Challenges 1-3
1:15 - 2:00	Challenges 4-6 (6 is a good take-home)

Suggested demos and live coding

- **The invisible leak.** In `ch06_hud_events/src/scenes/HudScene.ts` , delete the `this.events.once(Phaser.Scenes.Events.SHUTDOWN, ...)` block, rebuild, and press H a few times. There is **no error and the numbers are right** - but the "listeners" counts go 2, 3, 4. Ask why it still works (the scene object is reused, so old listeners call the same `HudScene` , whose fields now point at the new texts). Then ask: what if `showCoins` also played a sound?
- **Late listeners.** In `ch06_hud_events` , press 2 a few times, then H: `Gems: ?` . Ask why coins and stars survive a HUD restart and gems do not. Lead them to "events are news, not state".
- **Forgetting** `this.scene.stop(HUD_SCENE)` . Remove that line from `GameScene.endGame()` in `ch06_coin_collector` : the HUD bar stays on top of the game over screen (checked in a scratch copy: the active scenes were `HudScene` , `GameOverScene`). Pressing SPACE then relaunches the HUD, which *restarts* it - so it looks fine again, hiding the bug.
- **Why validate?** Show `ch06_high_score_table` starting normally with a corrupt value in storage, then with the `try / catch` removed: a blank screen and a `SyntaxError` in the console.

Key points to stress

- **Tell, then listen.** Scenes call the `ScoreManager`'s methods to report what happened; they never set the score. The `ScoreManager` emits events and never knows who listens. Most students' first design has the game scene updating HUD text directly, or the HUD reading game-scene fields every frame - both work, and both are worth comparing to this
- **Events are news, not state.** Anything that starts later (a launched HUD starts a frame late) must read the current values as well as listen
- **Clean up what you add to someone else's emitter.** Phaser clears a scene's own objects, timers and input listeners on shutdown. It does *not* remove listeners the scene put on the game, the registry, another scene, or a `ScoreManager`
- `on(event, fn, this)` - the third argument matters, and `off` needs the same three things
- **Never trust stored data.** Storage is shared by the whole site (every project in this book is served from 127.0.0.1:8000), can be edited by the player, and outlives versions of your game
- a new `ScoreManager` per game is the cleanest reset there is - contrast with Chapter 2's `init()`

Common problems and errors

These messages were produced by making each mistake in a scratch copy of the chapter projects.

What students see	Cause	Fix
Nothing wrong, but <code>listenerCount</code> climbs; effects play twice after a restart	listeners on another emitter not removed on <code>SHUTDOWN</code>	<code>off(...)</code> them in a <code>SHUTDOWN</code> handler
<code>TypeError: Cannot read properties of undefined (reading 'addCounter')</code> when the score changes	<code>this.scoreManager.on(SCORE_CHANGED, this.countUpTo)</code> - no context, so <code>this</code> is the <code>ScoreManager</code>	pass <code>this</code> as the third argument to <code>off()</code>
Argument of type <code>'number null'</code> is not assignable to parameter of type <code>'number'</code> .	<code>this.showScore(tween.getValue())</code> in a counter tween	<code>tween.getValue() ?? 0</code>
Argument of type <code>'string null'</code> is not assignable to parameter of type <code>'string'</code> .	<code>JSON.parse(localStorage.getItem("scores"))</code>	check for <code>null</code> first
Type <code>'Scene'</code> is missing the following properties from type <code>'GameScene'</code> : <code>coins</code> , <code>gems</code> , <code>preload</code> , <code>init</code> , and 6 more.	<code>this.gameScene = this.scene.get(GAME_SCENE);</code> into a <code>GameScene</code> field	<code>this.scene.get<GameScene>(GAME_SCENE)</code>

What students see	Cause	Fix
Blank screen; console: <code>SyntaxError: Unexpected token 'h', "this is not json" is not valid JSON</code>	<code>JSON.parse</code> with no <code>try / catch</code> , and bad data in storage	<code>try / catch</code> , and start table
The HUD never updates from the registry, or misses the first value	listening for <code>changedata-score</code> when the key is new: the first <code>set</code> sends <code>setdata</code>	set the key before anything <code>GameScene.init()</code> does the current value when it starts
The HUD restarts (score display resets) whenever the game scene restarts	<code>this.scene.launch(HUD_SCENE)</code> on a HUD that is already running restarts it	<code>if (!this.scene.isActive)</code>
HUD bar still showing over the game over screen	no <code>this.scene.stop(HUD_SCENE)</code> at game over	stop it before starting the
The old "new" row still flashes on the title screen after a later game	<code>this.scene.start(TITLE_SCENE)</code> with no data reuses the data it was last started with	pass <code>{}</code>
The initials screen types an extra letter, or the confirm prompt (challenge 4) cancels itself	a <code>keydown</code> listener added inside another key's listener receives the same key press	one <code>keydown</code> listener with
Scores "disappear"	playing in Celbridge's <code>game.webview</code> then in Chrome (or the other way round) - each browser has its own storage	expected; explain

What students see	Cause	Fix
Scores from another project appear, or vanish	two games using the same storage key on 127.0.0.1:8000	a key that names the gai

Discussion questions

- The ScoreManager extends `Phaser.Events.EventEmitter`. It could instead *have* one: `public readonly events = new Phaser.Events.EventEmitter();` - which is how `this.registry.events` works. What changes for the callers? Which do you prefer, and why? (Inheritance exposes `emit` to everyone, so any class can fake a `SCORE_CHANGED`; composition could keep `emit` private-ish.)
- `emit` is not type-checked. How could we make `SCORE_CHANGED`'s arguments checked by the compiler? (Typed wrapper methods such as `onScoreChanged(fn: (score: number) => void)`.)
- The game scene plays the level-up sound; the HUD shows the level-up banner. Both listen to `LEVEL_CHANGED`. Should the ScoreManager play the sound? Why not?
- Where should "best score this session" live - the registry, the ScoreManager, or the table?
- The high score table is loaded afresh by every scene that needs it, rather than kept in memory. What are the advantages? When would it be a bad idea?
- What could a player do to cheat the table? Can a browser game ever stop that? (No - anything in the browser can be edited. Only a server can keep honest scores.)

Extension ideas

- a pause scene (Chapter 4) that shows the ScoreManager's numbers - a third listener, for free
- unit-test the scoring rules: extract them into a class that does not extend a Phaser class (take an emitter in the constructor, or return results), and test it with `deno test`

- save the whole game's settings (volume from Chapter 5, difficulty from challenge 6) as one JSON object under one key, with a `version` number and a migration when the shape changes
 - show the table with a stagger tween, rows sliding in one after another (a preview of Chapter 7)
-

Assessment ideas

- Practical: "add a *time bonus*: points for every second survived, shown in the HUD, without the HUD reading anything from the game scene"
 - Code reading: show a HUD `create()` with `on` calls and no `SHUTDOWN` handler; ask what goes wrong, when, and how they would prove it (listener counts)
 - Short answer: what is the difference between `unknown` and `any`, and why is `JSON.parse`'s result better treated as `unknown`?
 - Design: draw the "tells / announces" diagram for a game of their choice (lives, ammo, keys)
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. A star is born

Project: `solutions/ch06_challenge_1_a_star_is_born` (from `ch06_coin_collector`)

Goal: find the places a pickup type is described, and add one - data, not new code.

`src/objects/Coin.ts`

```
export const GEM: CoinKind = { texture: GEM_KEY, points: 50, scale: 1.5,
lifetimeScale: 0.6 };
// CHALLENGE 1 - the rarest and quickest pickup
export const STAR: CoinKind = { texture: STAR_KEY, points: 100, scale: 1.6,
lifetimeScale: 0.45 };
```

`src/scenes/GameScene.ts`

```
const STAR_CHANCE = 0.05;           // CHALLENGE 1 - one pickup in twenty is
a star
...
// CHALLENGE 1 - one random number, split into three ranges: star, gem, coin
const roll = Math.random();
let kind = GOLD_COIN;
if (roll < STAR_CHANCE) {
    kind = STAR;
} else if (roll < STAR_CHANCE + GEM_CHANCE) {
    kind = GEM;
}
```

plus `STAR_KEY` / `STAR_FILE` in `assets.ts`, `this.load.image(STAR_KEY, STAR_FILE)` in `TitleScene.preload()`, a third picture on the title screen, and `star.png` copied into `public/assets/images/`.

Look for: `ScoreManager` , `HudScene` and `Coin` 's class untouched - the points come from the `CoinKind` , and the `ScoreManager` only ever sees a number. The most common slip is calling `Math.random()` twice (`if (Math.random() < 0.05) ... else if (Math.random() < 0.12)`), which quietly changes the odds: the gem chance becomes 12% of the remaining 95%. One roll, split into ranges, is the clean answer. Tested with the harness: 200 spawned pickups gave 167 coins, 24 gems and 9 stars, and a clicked star scored 100.

2. Bonus life

Project: `solutions/ch06_challenge_2_bonus_life` (from `ch06_coin_collector`)

Goal: see the payoff of events: change the rules in one place, and the display follows.

`src/ScoreManager.ts`

```
if (this.pickups % PICKUPS_PER_LEVEL === 0) {
    this.level = this.level + 1;
    this.emit(LEVEL_CHANGED, this.level);

    // CHALLENGE 2 - a new level gives back one lost life. The HUD already
    // listens for
    // LIVES_CHANGED, so it fills a heart in again without being changed at
    // all.
    if (this.lives < START_LIVES) {
        this.lives = this.lives + 1;
        this.emit(LIVES_CHANGED, this.lives);
    }
}
```

Why it works without touching the HUD (the question in the challenge): the HUD does not know *why* lives change. It listens for `LIVES_CHANGED` and redraws the hearts from the number it is given - so a life gained is shown exactly like a life lost.

Look for: the change in `ScoreManager` , not in `GameScene` (a student who adds a life in the game scene's `LEVEL_CHANGED` listener has to reach into the `ScoreManager` - there is no setter - and usually ends up adding one, which is worth discussing). The cap at three.

Tested with the harness: two lives lost, ten pickups - hearts go from 1, 0.2, 0.2 to 1, 1, 0.2 ; two more levels and all three are back, and no more.

3. Combo meter

Project: `solutions/ch06_challenge_3_combo_meter` (from `ch06_coin_collector`)

Goal: a new view of existing data, entirely in the HUD.

`src/ScoreManager.ts`

```
// CHALLENGE 3 - exported, so the HUD's meter can use the same numbers as  
the rules  
export const COMBO_STEP = 5;    // the multiplier goes up by 1 for every 5  
pickups in a row  
export const MAX_MULTIPLIER = 5;
```

`src/scenes/HudScene.ts`

```

this.add.rectangle(METER_X, METER_Y, METER_WIDTH, METER_HEIGHT, 0x000000,
0.6).setOrigin(0, 0.5);
this.meterFill = this.add.rectangle(METER_X, METER_Y, METER_WIDTH,
METER_HEIGHT, 0xf4a261).setOrigin(0, 0.5);
this.meterFill.scaleX = 0;
...
private showMeter(combo: number, multiplier: number): void {
    this.tweens.killTweensOf(this.meterFill);

    if (multiplier >= MAX_MULTIPLIER) {
        this.meterFill.scaleX = 1;    // the top multiplier: nothing more to
fill, so it stays full
        return;
    }

    const step = combo % COMBO_STEP;    // 0 to 4: pickups since the multiplier
last went up
    if (combo > 0 && step === 0) {
        // the multiplier has just gone up: show the bar full for a moment, then
empty it
        this.meterFill.scaleX = 1;
        this.tweens.add({ targets: this.meterFill, scaleX: 0, delay: 150,
duration: 300 });
    } else {
        // a fifth for each pickup - or empty when the combo is broken (combo 0,
step 0)
        this.tweens.add({ targets: this.meterFill, scaleX: step / COMBO_STEP,
duration: 120 });
    }
}

```

`showMeter` is called at the end of `showCombo`, which already runs on every `COMBO_CHANGED`.

- the fill is a `Rectangle` stretched with `scaleX`; origin `(0, 0.5)` makes it grow from the left. Changing `width` also works, but a `Rectangle`'s `width` does not redraw its shape - `setSize` does
- `%` (remainder) turns "combo 13" into "3 towards the next step"

Look for: the constants **exported** from `ScoreManager.ts`, not copied as a second `5` in the HUD (a copy goes wrong the day someone changes the rule). `killTweensOf` before starting a new tween, so an old one does not fight it. Accept a meter with no tween at all.

Tested with the harness: 3 pickups gives 0.6; the 5th fills it and it empties; a broken combo gives 0; the top multiplier keeps it full.

4. Wipe the slate

Project: `solutions/ch06_challenge_4_wipe_the_slate` (from `ch06_high_score_table`)

Goal: a small state machine for keyboard input, and removing something from storage.

`src/HighScoreTable.ts`

```
// CHALLENGE 4 - empty the table, and forget the saved copy too
public clear(): void {
  this.entries = [];
  try {
    localStorage.removeItem(STORAGE_KEY);
  } catch {
    // storage turned off: there was nothing saved to remove
  }
}
```

`src/scenes/TitleScene.ts`

```
// CHALLENGE 4 - R asks the question; then Y clears the table, and any other
key cancels
private handleKey(event: KeyboardEvent): void {
  const key = event.key.toLowerCase();
  if (!this.confirming) {
    if (key === "r") {
      this.confirming = true;
      this.prompt.setVisible(true);
    }
    return;
  }

  if (key === "y") {
    HighScoreTable.load().clear();
    const data: TitleData = {}; // no row to highlight any more
    this.scene.restart(data); // start this scene again: it reads the
    (now empty) table
  } else {
    this.confirming = false;
    this.prompt.setVisible(false);
  }
}
```

and the SPACE listener becomes `on` with a check, so it does nothing while the question is showing:

```
this.input.keyboard!.on("keydown-SPACE", () => {
  if (!this.confirming) {
    this.scene.start(GAME_SCENE);
  }
});
```

The trap most students fall into: listening for R, and then adding a `"keydown"` listener *inside* the R listener to hear the answer. Phaser sends `keydown-R` first and then `keydown` for the **same** key press - so the brand-new listener hears the R, and takes it as the answer. We checked this in a scratch copy: the answer listener logged `answer: r` straight after `asking`. One `keydown` listener with a `confirming` flag avoids it.

Look for: `confirming` reset in `init()` (the scene object is reused); `clear()` on `HighScoreTable` rather than `localStorage.removeItem` with the key copied into the scene; `restart({})` rather than `restart()` - without data, the new start could still carry

the old `highlight` . Tested with the harness: R shows the question, N or SPACE cancel and leave the table, R then Y empties it, the storage key is gone, and it is still empty after a reload.

5. Chasing the record

Project: `solutions/ch06_challenge_5_chasing_the_record` (from `ch06_high_score_table`)

Goal: a second listener on the same event, with its own state.

`src/scenes/HudScene.ts`

```

init(data: HudData): void {
    ...
    // CHALLENGE 5 - the HUD reads the table itself: it is the only one that
    needs the top score
    this.record = HighScoreTable.load().getTopScore();
    this.celebrated = false;
}
...
this.scoreManager.on(SCORE_CHANGED, this.checkRecord, this); // CHALLENGE
5 - a second listener
...
private showScore(score: number): void {
    this.shownScore = score;
    this.scoreText.setText(String(score).padStart(SCORE_DIGITS, "0"));
    // CHALLENGE 5 - HI shows the record, or the score on screen once that is
    higher, so the two
    // count up together
    this.hiText.setText(String(Math.max(this.record,
score)).padStart(SCORE_DIGITS, "0"));
}

private checkRecord(score: number): void {
    if (this.celebrated || this.record === 0 || score <= this.record) {
        return;
    }
    this.celebrated = true;
    this.hiText.setColor("#ffd166");
    ...
}

```

The rest of `checkRecord` makes a big "NEW HIGH SCORE!" text and blinks it with a tween before destroying it (the same pattern as the level banner).

Look for:

- `celebrated` reset in `init()` - otherwise the second game never celebrates (Chapter 2's trap)
- the matching `off` for the new listener in the `SHUTDOWN` handler
- `HI` updated in `showScore` (so it counts up with the score) rather than jumping to the new score in the listener - both are acceptable, but ask which looks better
- `record === 0` : with an empty table every first point would be a "new high score". A good discussion point - is it one?

Some students put the record in the `ScoreManager`. That is defensible (it is a scoring fact), but then the `ScoreManager` needs the table, or has to be given the record - ask them to argue for it. Tested with the harness: record 300; at 170 and 230 nothing; at 330 `HI` shows 000330, gold, and the message appears once.

6. Easy, normal, hard

Project: `solutions/ch06_challenge_6_easy_normal_hard` (from `ch06_high_score_table`)

Goal: data that travels through every scene; one storage key per table.

A new file describes the difficulties:

`src/Difficulty.ts`

```
export type Difficulty = "easy" | "normal" | "hard";

export interface DifficultySettings {
  name: string;
  lifetimeScale: number; // how long coins stay: 1.4 = 40% longer
  spawnScale: number;   // the time between coins: 0.75 = they come a
  // quarter faster
}

export const DIFFICULTIES: Record<Difficulty, DifficultySettings> = {
  easy: { name: "EASY", lifetimeScale: 1.4, spawnScale: 1.25 },
  normal: { name: "NORMAL", lifetimeScale: 1, spawnScale: 1 },
  hard: { name: "HARD", lifetimeScale: 0.7, spawnScale: 0.75 },
};
```

The answer to the hint's question - `load()` takes the difficulty, and uses it to choose the storage key; the table remembers its key for `save()` :

`src/HighScoreTable.ts`

```

// CHALLENGE 6 - each difficulty is saved under its own key. "normal" keeps
the original key, so
// the scores saved before there were difficulties still count as normal
ones.
function storageKeyFor(difficulty: Difficulty): string {
    return difficulty === "normal" ? STORAGE_KEY :
`${STORAGE_KEY}.${difficulty}`;
}
...
public static load(difficulty: Difficulty = DEFAULT_DIFFICULTY):
HighScoreTable {
    const key = storageKeyFor(difficulty);

```

Then the difficulty is carried in scene data all the way round: `TitleData` and a new `GameData` gain `difficulty?`, `GameOverData` gains `difficulty`, and every `this.scene.start` passes it on. The title screen's 1, 2 and 3 keys restart it with the chosen difficulty:

`src/scenes/TitleScene.ts`

```

private choose(difficulty: Difficulty): void {
    const data: TitleData = { difficulty: difficulty };
    this.scene.restart(data);
}

```

and the game scene scales its timings:

`src/scenes/GameScene.ts`

```

const lifetime = levelLifetime *
DIFFICULTIES[this.difficulty].lifetimeScale; // CHALLENGE 6

```

Look for:

- a union type (or enum) for the difficulty rather than bare strings or numbers - then `Record<Difficulty, ...>` makes the compiler insist every difficulty has settings
- the difficulty in **every** hand-over. The usual bug: playing again from the game over screen (SPACE) silently goes back to normal, because `GameOverScene` starts `GameScene` with no data. The tidy fix is the one here; storing the current difficulty in the registry is also a fair answer

- keeping the old key for normal - a thoughtful touch, not required
- a default parameter (`difficulty: Difficulty = DEFAULT_DIFFICULTY`) so older calls still compile

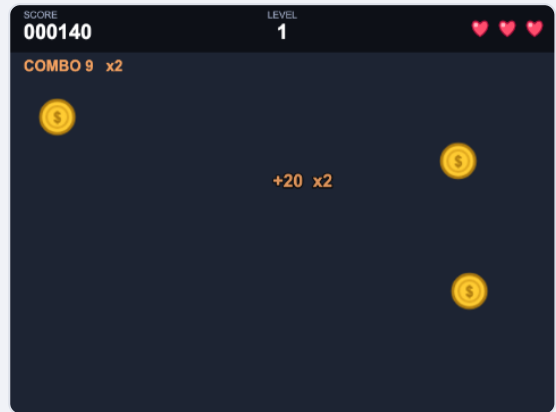
Tested with the harness: a seeded normal table shows under NORMAL; 3 shows an empty HARD table; a hard game's coins last 1680 ms instead of 2400; a hard high score goes into `phaser4-guide.coin-collector.high-scores.hard` and the normal table is untouched; after an easy game, SPACE on the game over screen plays easy again.

Slides

Chapter 6

Scoring

A HUD, a ScoreManager, and high scores that last



1 / 20

Today

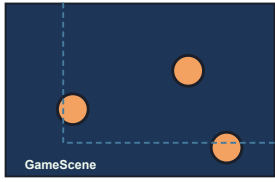
- a **HUD** in a scene of its own, on top of the game
- three ways to tell the HUD something changed
- a **ScoreManager**: tell it what happened; it announces the results
- floating "+10"s and a score that counts up
- high scores saved in the browser - and checked when they come back
- typing initials

2 / 20

A HUD is a scene

Two scenes running at once, one drawn on top of the other

HudScene (see-through, except its bar)



GameScene

main.ts

```
scene: [TitleScene, GameScene,
        HudScene, GameOverScene]

Later in the list = drawn later = on top.
```

GameScene

```
this.scene.launch(HUD_SCENE, data)
starts the HUD beside it, on the next frame

this.scene.stop(HUD_SCENE)
takes it away again when the game ends
```

```
this.scene.launch(HUD_SCENE, hudData); // beside this scene, from the next frame
...
this.scene.stop(HUD_SCENE);           // when the game ends
```

3 / 20

Why a separate scene?

- **its own camera** - `this.cameras.main.shake(...)` in the game does not shake the score
- draw order = order in the config's `scene` list: later is on top
- the game scene can restart without the HUD rebuilding itself
- one job per class: the game plays, the HUD shows

4 / 20

Emitters

An **event emitter**: `emit` events on it; listen with `on` , `once` , `off`

```
ball.on("pointerdown", ...) // you have used one already
```

Three that can reach a HUD:

- the game scene's own: `this.events`
- the game's: `this.game.events`
- the registry's: `this.registry.events`

5 / 20

Try it: HUD Events

- click the coin, gem and star (or 1, 2, 3)
- **R** restarts the game scene; **H** restarts the HUD
- watch the "listeners" counts
- after H, why does it say `Gems: ?`



6 / 20

Three routes

```
// 1 - the game scene's own emitter (the HUD needs the scene)
this.events.emit(COINS_CHANGED, this.coins);
this.gameScene.events.on(COINS_CHANGED, this.showCoins, this);

// 2 - the game's emitter (anyone can send, anyone can listen)
this.game.events.emit(GEMS_CHANGED, this.gems);
this.game.events.on(GEMS_CHANGED, this.showGems, this);

// 3 - the registry announces its own changes
this.registry.inc(STARS, 1);
this.registry.events.on(`changedata-${STARS}`, this.onStarsChanged, this);
```

First `set` of a new key sends `setdata`, not `changedata`

7 / 20

News and state

- an event is **news**: only listeners there *at the time* hear it
- a launched HUD starts a frame late - so it also reads the current values

```
this.showCoins(this.gameScene.getCoins()); // route 1: ask the scene
this.gemsText.setText("Gems: ?"); // route 2: nobody to ask
this.showStars(this.registry.get(STARS)); // route 3: registry = state + news
```

8 / 20

Clean up

```
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  this.gameScene.events.off(COINS_CHANGED, this.showCoins, this);
  this.game.events.off(GEMS_CHANGED, this.showGems, this);
  this.registry.events.off(`changedata-${STARS}`, this.onStarsChanged, this);
});
```

- those emitters **outlive the HUD** - Phaser will not remove these listeners
- forget it: **no error**, the listeners just pile up
- `off` needs the same event, function **and context** as `on`

9 / 20

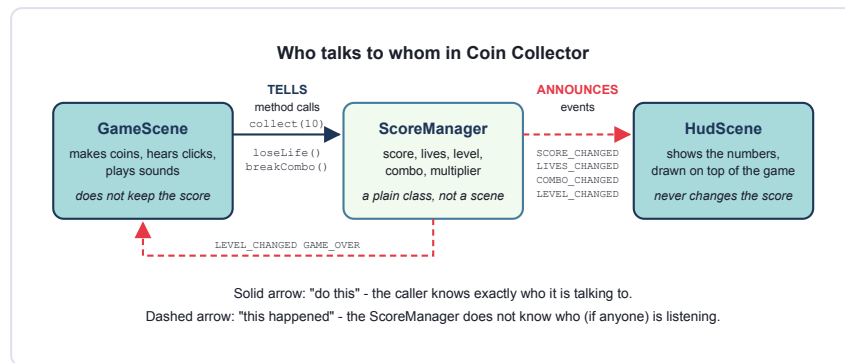
A ScoreManager

```
export class ScoreManager extends Phaser.Events.EventEmitter {
  private score = 0;
  private lives = START_LIVES;
  ...
  public collect(basePoints: number): number {
    this.combo = this.combo + 1;
    const points = basePoints * this.getMultiplier();
    this.score = this.score + points;
    this.emit(SCORE_CHANGED, this.score, points);
    ...
  }
}
```

A plain class: no text, no sounds, no scenes

10 / 20

Tell - announce



The observer pattern - Swing's `addActionListener`

11 / 20

The HUD listens

```
init(data: HudData): void {
    this.scoreManager = data.scoreManager;
    ...
}

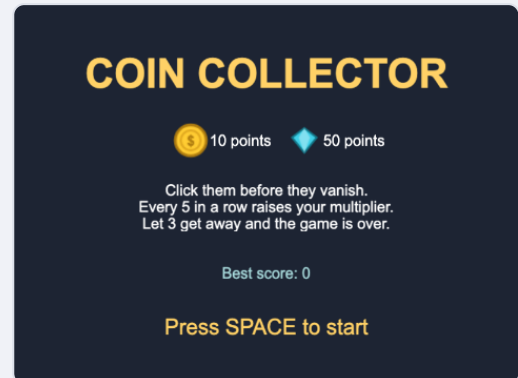
create(): void {
    ...
    this.showScore(this.scoreManager.getScore()); // now
    this.scoreManager.on(SCORE_CHANGED, this.countUpTo, this); // and later
    ...
}
```

New game? `this.scoreManager = new ScoreManager();` - nothing to reset

12 / 20

Try it: Coin Collector

- 5 in a row raises the multiplier
- 10 pickups = a faster level
- 3 escape = game over
- find every `emit` and every `on`



13 / 20

Feel: float and count

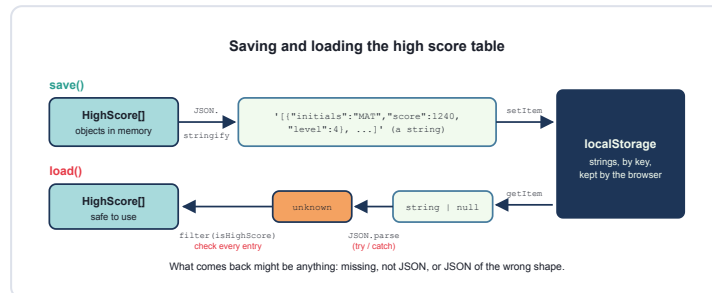
```
// FloatingText: rise, fade, then destroy itself
scene tweens.add({ targets: this, y: y - RISE, alpha: { value: 0, ease: "Cubic.easeIn" },
  duration: DURATION, ease: "Cubic.easeOut", onComplete: () => this.destroy() });

// HudScene.countUpTo(score)
this.countUp?.stop();
this.countUp = this.tweens.addCounter({
  from: this.shownScore, to: score, duration: COUNT_UP_TIME, ease: "Cubic.easeOut",
  onUpdate: (tween) => {
    this.showScore(Math.round(tween.getValue() ?? 0));
  },
});
```

14 / 20

localStorage and JSON

- `setItem(key, text)`, `getItem(key)` (or `null`), `removeItem(key)`
- **strings only** - `JSON.stringify` / `JSON.parse`
- kept **per web site**: every project on 127.0.0.1:8000 shares one - name your key
- each browser has its own (Celbridge's webview is not Chrome)



15 / 20

Never trust what comes back

```
let data: unknown;
try {
  data = JSON.parse(text);
} catch {
  return new HighScoreTable([]); // not JSON at all
}
if (!Array.isArray(data)) {
  return new HighScoreTable([]); // JSON, but not a list
}
const entries = data.filter(HighScoreTable.isHighScore);
```

`unknown` : check before use. `isHighScore(value: unknown): value is HighScore` - a **type guard**

16 / 20

Try it: High Score Table

- get onto the table; find the key in the developer tools
- change it to `this is not json` and reload
- what happens to a score that *ties*?

COIN COLLECTOR			
HIGH SCORES			
1.	ACE	002450	level 7
2.	BOB	001980	level 6
3.	CAT	001620	level 5
4.	DAN	001400	level 5
5.	MAT	001240	level 4
6.	EVE	001130	level 4
7.	FOX	000960	level 4
8.	GUS	000720	level 3
9.	HAL	000540	level 3
10.	IVY	000350	level 2

Press SPACE to start

Click coins before they vanish. 5 in a row raises the multiplier.

17 / 20

Typing initials

```
this.input.keyboard!.on("keydown", (event: KeyboardEvent) => {
  this.handleKey(event);
});
...
if (LETTER.test(event.key) && this.initials.length < MAX_INITIALS) {
  this.initials = this.initials + event.key.toUpperCase();
} else if (event.key === "Backspace") {
  this.initials = this.initials.slice(0, -1);
}
```

`event.key` - what the key types. `event.code` - where the key is

18 / 20

Summary

- a HUD is a scene, launched on top; it has its own camera
- emitters: `this.events` , `this.game.events` , `this.registry.events` , your own
- events are news: read the current state too, and clean up on `SHUTDOWN`
- scenes **tell** the ScoreManager; it **announces**
- tweens and counter tweens make scores feel good
- `localStorage` + JSON for scores that last - validated on the way back in

19 / 20

Challenges

1. **A star is born** - a rare 100-point star
2. **Bonus life** - a new level gives a life back; the HUD needs no change
3. **Combo meter** - a bar that fills towards the next multiplier
4. **Wipe the slate** - R, then Y, clears the table
5. **Chasing the record** - `HI` in the HUD, and a celebration
6. **Easy, normal, hard** - a table for each difficulty

Next: **Chapter 7 - Animations**

20 / 20